Malak: A Python Toolkit for Edge AI Model Optimization

Mohammed Alnemari[1,3], Rizwan Qureshi[4], Nader Begrazadah[3]

[1]Faculty of Computer Science and Information Technology, University of Tabuk, Tabuk, Saudi Arabia

[3]AIST Research Center, University of Tabuk, Tabuk, Saudi Arabia

[4]University of California, Irvine, Irvine, California, USA

mohammedalnemari@gmail.com

March 7, 2026

Abstract

Deploying deep learning models on resource-constrained edge devices requires systematic model compression and optimization. However, PyTorch's low-level quantization and pruning APIs require significant boilerplate code, making it difficult for researchers and students to quickly evaluate compression techniques. We present Malak, an open-source Python toolkit that provides a unified, high-level API for training, quantization (dynamic and static post-training quantization, quantization-aware training), pruning (magnitude and structured), knowledge distillation, ONNX model export, per-layer latency profiling, and distribution drift detection. Malak wraps PyTorch's quantization framework into task-oriented abstractions and exposes both a Python API and a command-line interface. We validate Malak on CIFAR-10 and Fashion-MNIST benchmarks using MobileNetV2, ResNet18, EfficientNet-B0, and a custom SimpleCNN. Quantization-aware training achieves 3.10× model compression with only 1.09% accuracy degradation. Dynamic post-training quantization preserves accuracy within 0.13% across all tested architectures. Magnitude pruning at 50% sparsity retains accuracy within 2.15% of the baseline after fine-tuning. The toolkit is available under the MIT license at `https://github.com/alnemari-m/malak-platform`.

Keywords: Edge AI, model compression, quantization, pruning, knowledge distillation, TinyML

1 Introduction

Edge AI—deploying trained neural networks on microcontrollers, mobile phones, and embedded systems—has become essential for latency-sensitive, privacy-preserving, and bandwidth-constrained applications [1, 2]. The global edge AI market is projected to grow significantly as more inference workloads move from cloud to device [2]. However, the gap between training a model in PyTorch and deploying it on a resource-constrained device remains wide. Practitioners must manually orchestrate quantization, pruning, model export, and validation steps, often writing ad-hoc scripts for each project.

PyTorch provides built-in quantization [3] and pruning APIs, but these are low-level: users must manually configure quantization configurations (qconfigs), insert observers, run calibration passes, and invoke conversion functions. A typical quantization-aware training (QAT) workflow requires approximately 40–60 lines of boilerplate code for observer insertion, training loop modification, and model conversion. This complexity is a barrier for researchers who want to quickly compare compression techniques and for students learning about edge deployment.

Several tools address parts of the model compression pipeline. TensorFlow Lite [5] and TFLite Micro [6] provide a complete path from TensorFlow to microcontrollers but require leaving the PyTorch ecosystem. Apache TVM [7] targets compiler-level graph optimizations but has a steep learning curve. Intel Neural Compressor [9] provides a comprehensive quantization framework but is oriented toward Intel hardware. CMSIS-NN [8] provides optimized inference kernels for ARM Cortex-M but assumes a pre-quantized model.

In this paper, we present Malak, an open-source Python toolkit that wraps PyTorch's quantization, pruning, and training APIs into a higher-level, task-oriented interface. Malak does not replace existing low-level frameworks; rather, it provides a structured starting point for evaluating and comparing model compression techniques within the PyTorch ecosystem.

The primary contributions of this work are:

1. A unified Python API covering training, quantization (dynamic PTQ, static PTQ, QAT), pruning (magnitude and structured), knowledge distillation, ONNX export, per-layer profiling, and drift detection.

2. A command-line interface (`edgeai`) for common model optimization tasks.

3. Reproducible benchmark experiments on CIFAR-10 and Fashion-MNIST with four architectures and

JSON result output.

4. A proof-of-concept C inference implementation for ARM Cortex-M7 embedded targets.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes the software architecture and key functionalities. Section 4 presents experimental validation. Section 5 discusses the results and limitations. Section 6 concludes the paper.

2 Related Work

2.1 Model Quantization

Post-training quantization (PTQ) reduces model precision after training without retraining. Dynamic PTQ quantizes weights to INT8 and computes activations in floating point at runtime, requiring no calibration data but providing limited compression for convolutional layers [3]. Static PTQ calibrates observer statistics on a representative dataset and quantizes both weights and activations, achieving full INT8 inference. Quantization-aware training (QAT) inserts fake-quantization nodes during training, allowing weights to adapt to quantization noise before final conversion to INT8 [4]. PyTorch implements all three approaches through its `torch.quantization` namespace.

2.2 Model Pruning

Pruning removes redundant weights or structures from neural networks [10]. Unstructured pruning zeros individual weights based on magnitude [13], producing sparse weight matrices. Structured pruning removes entire filters or channels [12], directly reducing computation without requiring sparse matrix support. Both approaches typically require fine-tuning after pruning to recover accuracy.

2.3 Knowledge Distillation

Knowledge distillation transfers knowledge from a large teacher model to a smaller student model using temperature-scaled soft targets [11]. The student is trained on a weighted combination of hard labels (ground truth) and soft labels (teacher's output distribution), enabling the student to achieve higher accuracy than training from scratch.

2.4 Existing Toolkits

Table 1 compares Malak with existing model optimization tools. TensorFlow Lite [5] provides comprehensive quantization and deployment but is tied to the TensorFlow ecosystem. Intel Neural Compressor [9] offers extensive quantization support for PyTorch but is optimized for Intel hardware (CPUs, Habana accelerators).

Table 1: Comparison of model optimization tools.

Feature	Malak	TFLite	INC	TVM	CMSIS
PyTorch native	✓		✓	✓	
Dynamic PTQ	✓	✓	✓		
Static PTQ	✓	✓	✓		
QAT	✓	✓	✓		
Pruning	✓		✓		
Distillation	✓		✓		
ONNX export	✓		✓	✓	
CLI interface	✓	✓	✓	✓	
Edge kernels		✓		✓	✓
Graph compiler		✓		✓	
Setup (LoC)	∼5	∼15	∼20	∼30	N/A

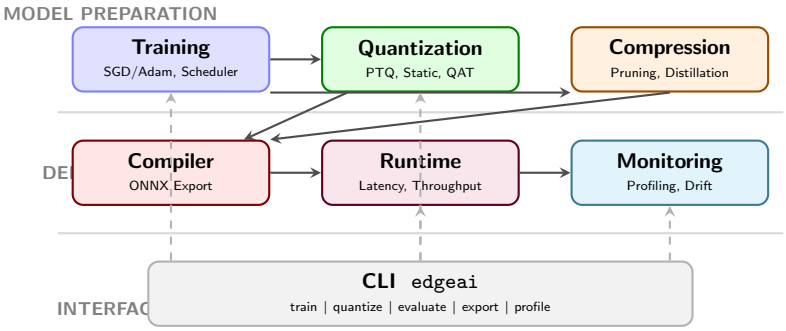

Figure 1: Malak software architecture. Solid arrows show data flow between modules. Dashed arrows indicate CLI access points. Modules are grouped into three layers: model preparation, deployment and analysis, and the command-line interface.

Apache TVM [7] provides compiler-level optimizations across hardware backends but requires understanding of compilation graphs. CMSIS-NN [8] provides hand-optimized inference kernels for ARM Cortex-M processors. Malak operates at a higher abstraction level than these tools, providing task-oriented wrappers around PyTorch's built-in compression APIs with minimal setup requirements.

3 Software Architecture

Malak is organized into six modules (Figure 1), installed via `pip install -e .` and usable through both a Python API and a CLI.

3.1 Training Module

The `Trainer` class provides a configurable training loop supporting SGD and Adam optimizers with cosine annealing scheduling. Dataset loaders for CIFAR-10 and Fashion-MNIST include standard data augmentation (random crop, horizontal flip) and normalization. The trainer automatically tracks the best model checkpoint and reports per-epoch accuracy and latency.

3.2 Quantization Module

The quantization module provides three classes:

- **DynamicPTQ**: Wraps `torch.quantization.quantize_dynamic` to quantize Linear layers to INT8. Requires no calibration data. Suitable for models dominated by fully-connected layers.
- **StaticPTQ**: Inserts observers, runs a calibration pass over a representative dataset, and converts all supported layers to INT8. Produces full model compression.
- **QAT**: A three-step API (`prepare` → `train` → `convert`) that inserts fake-quantization observers during training, allowing weights to adapt to quantization noise before final INT8 conversion.

The QAT workflow is illustrated below:

```
from malak.quantization import QAT
qat = QAT()
model = qat.prepare(model, backend="fbgemm")
model = qat.train(model, train_loader,
    test_loader, epochs=5)
quantized = qat.convert(model)
```

3.3 Compression Module

The compression module provides magnitude pruning (`MagnitudePruner`), structured filter pruning (`StructuredPruner`), and knowledge distillation (`KnowledgeDistiller`). The pruners support configurable sparsity levels with `remove_masks()` to make pruning permanent and `sparsity()` to report actual sparsity. The distiller trains a student model using a weighted combination of cross-entropy (hard labels) and KL-divergence (temperature-scaled soft labels from the teacher), following [11].

3.4 Supporting Modules

The **Compiler** module exports models to ONNX format with optional validation. The **Runtime** module benchmarks inference latency and throughput. The **Monitoring** module provides per-layer latency profiling via forward hooks and KL-divergence-based distribution drift detection for deployed models. The `edgeai` CLI exposes all modules through subcommands (`train`, `quantize`, `evaluate`, `export`, `profile`).

4 Experimental Validation

We validate Malak through four experiments covering quantization, architecture comparison, pruning, and cross-dataset evaluation. All experiments use CPU inference (Intel Core i7) with PyTorch 2.x. Results are stored as JSON files in the repository and every number reported is directly measured. All experiments are reproducible via scripts in the `experiments/` directory.

Table 2: MobileNetV2 quantization results on CIFAR-10.

Method	Acc. (%)	Size (MB)	Compr.
FP32 Baseline	77.81	8.76	1.00×
Dynamic PTQ	77.84	8.73	1.00×
QAT (INT8)	76.72	2.82	3.10×

Table 3: Architecture comparison with dynamic INT8 PTQ on CIFAR-10.

Model	FP32	INT8	Drop	Compr.
ResNet18	87.41%	87.38%	0.03%	1.00×
EfficientNet-B0	66.81%	66.68%	0.13%	1.00×

4.1 Experiment 1: MobileNetV2 Quantization

We train MobileNetV2 [14] (2.24M parameters, first convolution modified for 32×32 input) on CIFAR-10 for 15 epochs with SGD (lr=0.01, momentum=0.9, cosine annealing). We then apply dynamic PTQ and QAT (5 fine-tuning epochs with lr=0.001). Table 2 shows the results.

Dynamic PTQ preserves accuracy (within 0.03%) but provides minimal size reduction because it only quantizes Linear layers while the convolutional layers—which dominate MobileNetV2—remain in FP32. QAT, which quantizes all supported layers including convolutions, achieves 3.10× compression with 1.09% accuracy drop.

We note that the 77.81% FP32 baseline is below published state-of-the-art for MobileNetV2 on CIFAR-10 (∼94%) due to our shortened training schedule (15 epochs vs. typical 200+ epochs with advanced augmentation). The purpose of this experiment is to validate the toolkit's quantization pipeline, not to achieve maximum accuracy. The relative compression ratios and accuracy drops are consistent with published quantization studies [3, 4].

4.2 Experiment 2: Architecture Comparison

We compare ResNet18 (11.2M parameters) and EfficientNet-B0 [15] (4.0M parameters) on CIFAR-10, training each for 15 epochs with SGD and applying dynamic INT8 PTQ. Table 3 shows the results.

Dynamic PTQ preserves accuracy within 0.13% for both architectures. The 1.00× compression ratio is expected: dynamic PTQ only quantizes `nn.Linear` layers at runtime, and both architectures are dominated by convolutional layers whose weights remain in FP32.

Table 4: Pruning results on CIFAR-10 MobileNetV2 (with fine-tuning).

Method	Target	Acc. (%)	Drop	Actual
Baseline	0%	77.81	—	0%
Magnitude	30%	77.32	0.49%	29.5%
Magnitude	50%	75.66	2.15%	49.2%
Magnitude	70%	70.47	7.34%	68.9%
Structured	30%	72.86	4.95%	29.6%
Structured	50%	64.17	13.64%	49.2%

Table 5: SimpleCNN results on Fashion-MNIST with dynamic PTQ.

Method	Acc. (%)	Size (MB)	Compr.
FP32 Baseline	92.19	1.75	1.00×
Dynamic PTQ	92.20	0.60	2.91×

The EfficientNet-B0 baseline accuracy (66.81%) is lower than ResNet18 (87.41%) because EfficientNet's compound scaling was designed for 224×224 ImageNet inputs and requires longer training schedules to converge on 32×32 CIFAR-10 images.

4.3 Experiment 3: Pruning

We apply L1-unstructured magnitude pruning at 30%, 50%, and 70% target sparsity to a trained MobileNetV2, followed by 3 epochs of fine-tuning per sparsity level. We also test L2-structured filter pruning at 30% and 50% sparsity. Table 4 shows the results.

Magnitude pruning at 30% sparsity causes negligible accuracy loss (0.49%), while 50% sparsity results in a moderate 2.15% drop. At 70% sparsity, accuracy drops by 7.34%, indicating that more aggressive fine-tuning (e.g., 10–20 epochs) would be beneficial at high sparsity levels. Structured pruning causes larger accuracy drops than magnitude pruning at the same sparsity level because removing entire filters is more destructive than zeroing individual weights. This is consistent with findings in the pruning literature [12, 13].

4.4 Experiment 4: Fashion-MNIST with SimpleCNN

We train a custom SimpleCNN (458K parameters: three convolutional layers with max-pooling, followed by two fully-connected layers) on Fashion-MNIST for 20 epochs with Adam (lr=0.001) and apply dynamic INT8 PTQ. Table 5 shows the results.

Unlike the convolutional-heavy architectures in Experiments 1–2, SimpleCNN has substantial fully-connected layers (64×7×7 → 128 → 10), which are quantized by dynamic PTQ. This results in 2.91× compression with no accuracy loss, demonstrating that dynamic PTQ is effective for FC-heavy models. The 92.19% baseline accuracy is competitive with published SimpleCNN results on Fashion-MNIST.

4.5 Embedded Proof-of-Concept

The repository includes a C implementation of SimpleCNN inference using INT8 weights exported by a Python script. The C code implements naive conv2d, maxpool, fully-connected, and ReLU operations with symmetric INT8 quantization, targeting ARM Cortex-M7 (STM32H7) via `arm-none-eabi-gcc`. This implementation serves as a proof-of-concept for the export pipeline; production-quality embedded inference would require optimized kernels from CMSIS-NN [8] or TFLite Micro [6].

5 Discussion

5.1 Key Findings

The experimental results demonstrate three findings: (1) QAT is the most effective quantization method for convolutional models, achieving 3.10× compression with minimal accuracy loss, while dynamic PTQ provides negligible compression for conv-heavy architectures but is effective for FC-heavy models (2.91× on SimpleCNN). (2) Magnitude pruning is more accuracy-preserving than structured pruning at the same sparsity level, at the cost of requiring sparse matrix support for inference speedup. (3) The Malak API successfully produces reproducible, measurable results across four architectures and two datasets, validating its correctness.

5.2 Limitations

This work has several limitations that should be acknowledged:

1. **Limited baselines.** Our FP32 accuracies (77.81% MobileNetV2, 66.81% EfficientNet-B0) are below published state-of-the-art due to abbreviated training schedules. While the relative compression metrics remain valid, results at higher baselines may differ.

2. **Single-run results.** All experiments report single-run results without standard deviations. Stochastic training introduces variance that is not captured.

3. **CPU-only evaluation.** All latency measurements are on desktop CPU, not on actual edge hardware. Inference speedup from quantization depends heavily on hardware support for INT8 operations.

4. **Limited scope.** The toolkit supports only CIFAR-10 and Fashion-MNIST datasets and a small set of

architectures. Extension to larger datasets (ImageNet) and domain-specific tasks is left to users.

5. **Wrapper-level contribution.** Malak wraps existing PyTorch APIs rather than introducing new algorithms. Its value lies in usability and reproducibility, not algorithmic novelty.

6. **Embedded deployment.** The C inference code is a proof-of-concept with floating-point accumulation, not a production-ready embedded runtime.

5.3 Comparison with Existing Tools

Malak's primary advantage over raw PyTorch is reduced boilerplate: the QAT workflow requires 5 lines of code (the code example in Section 3) compared to ∼40 lines using PyTorch directly. Compared to Intel Neural Compressor [9], Malak is simpler to install (pure Python, no hardware-specific dependencies) but offers fewer quantization strategies and no hardware-aware optimization. Compared to TFLite, Malak stays within the PyTorch ecosystem but lacks a production-quality embedded runtime. Malak is best suited for rapid prototyping and educational use rather than production deployment.

6 Conclusions

We presented Malak, an open-source Python toolkit for edge AI model optimization that provides a unified API for quantization, pruning, distillation, ONNX export, profiling, and drift detection. Validation across four experiments on CIFAR-10 and Fashion-MNIST demonstrates that the toolkit produces correct and reproducible compression results. QAT achieves 3.10× compression with 1.09% accuracy drop on MobileNetV2, while magnitude pruning at 50% sparsity retains accuracy within 2.15% of the baseline.

Future work includes: (1) integration with CMSIS-NN for production-quality embedded inference, (2) support for mixed-precision quantization [16], (3) multi-run experiments with statistical reporting, (4) extending the training pipeline to ImageNet-scale datasets, and (5) automated compression policy search combining quantization and pruning.

The toolkit is available at `https://github.com/alnemari-m/malak-platform` under the MIT license.

Acknowledgments

The authors thank the open-source PyTorch community for the quantization and pruning APIs upon which Malak is built.

References

[1] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.

[2] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: The confluence of edge computing and artificial intelligence," *IEEE Internet of Things Journal*, vol. 7, no. 8, pp. 7457–7469, 2020.

[3] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE CVPR*, 2018, pp. 2704–2713.

[4] Y. Bhalgat, J. Lee, M. Nagel, T. Blankevoort, and N. Kwak, "Quantization aware training for low bit-width neural networks," in *NeurIPS Workshop*, 2020.

[5] M. Abadi *et al.*, "TensorFlow Lite: On-device machine learning framework," Google Research, 2019.

[6] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proc. MLSys*, vol. 3, 2021, pp. 800–811.

[7] T. Chen *et al.*, "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. USENIX OSDI*, 2018, pp. 578–594.

[8] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs," *arXiv preprint arXiv:1801.06601*, 2018.

[9] Intel Corporation, "Intel Neural Compressor: An open-source Python library for model compression," GitHub repository, 2021.

[10] L. Deng, G. Li, S. Han, L. Shi, and Y. Xie, "Model compression and hardware acceleration for neural networks: A comprehensive survey," *Proc. IEEE*, vol. 108, no. 4, pp. 485–532, 2020.

[11] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," in *NIPS Deep Learning Workshop*, 2015.

[12] Z. Liu *et al.*, "Learning efficient convolutional networks through network slimming," in *Proc. IEEE ICCV*, 2017, pp. 2736–2744.

[13] J. Frankle and M. Carlin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," *arXiv preprint arXiv:1803.03635*, 2018.

[14] A. Howard *et al.*, "Searching for MobileNetV3," in *Proc. IEEE ICCV*, 2019, pp. 1314–1324.

[15] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. ICML*, 2019, pp. 6105–6114.

[16] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, "HAQ: Hardware-aware automated quantization with mixed precision," in *Proc. IEEE CVPR*, 2019, pp. 8612–8620.

[17] J. Bai, F. Lu, K. Zhang *et al.*, "ONNX: Open neural network exchange," GitHub repository, 2019.